



LangChain4J-CDI

2026/05/27

MAKE IT / JCON SLOVENIA 2026

Buhake Sindi



Langchain4J-CDI

Generative AI
Natively Injectable
into Jakarta EE



Today 's program

-  **Architecture**
 - How it works under the hood
-  **@RegisterAIService**
 - Injection in action
-  **Progressive Features**
 - Memory, RAG, Tools, MCP
-  **Ready for production**
 - Resiliency (Fault Tolerance) and Observability (Telemetry)
-  **Live demos**
 - The full chatbot in production
-  **Guardrails**
 - Validate the entry and exit of your agents
-  **A2A: Agent-to-Agent**
 - Multi-agent orchestration via the A2A protocol



The problem

```
public class AiServicesDemo {  
  
    interface Assistant {  
        String chat(String message);  
    }  
  
    public static void main(String[] args) {  
        // With LangChain4j "vanilla"... it's verbose  
        ChatModel model = OpenAiChatModel.builder()  
            .apiKey(System.getenv("OPENAI_API_KEY"))  
            .modelName("gpt-4o")  
            .build();  
  
        ChatMemory chatMemory = MessageWindowChatMemory.withMaxMessages(20);  
        Assistant assistant = AiServices.builder(Assistant.class)  
            .chatLanguageModel(model)  
            .chatMemory(chatMemory)  
            .tools(new BookingTools())  
            .build();  
  
        String question = "Book me a table for the JCON Slovenia workshop.";  
        String answer = assistant.chat(question);  
        System.out.println("Answer: " + answer);  
    }  
}
```

- Not so much enterprise ready. 😬



The CDI solution..

```
Assistant.java

@registerAIService(chatModelName = "my-model", tools = BookingTools.class)
public interface Assistant {

    @SystemMessage("You are a booking assistant")
    String chat(@UserMessage String message);
}
```



In your business logic.



```
@Inject
```

```
Assistant assistant;
```

```
// That's it, really.
```

```
String response = assistant.chat("Book me a table");
```



Spring AI vs Langchain4J-CDI

☘ SPRING AI / SPRING BOOT 3.X

```
// Configuration bean
@Configuration
public class AssistantConfig {
    @Bean
    ChatClient chatClient(
        ChatClient.Builder b,
        VectorStore vs, ChatMemory mem) {
        return b.defaultSystem("You are helpful")
            .defaultAdvisors(
                new QuestionAnswerAdvisor(vs),
                new MessageChatMemoryAdvisor(mem))
            .defaultTools(new WeatherTools())
            .build();
    }
}
// Service that uses it
@Service
public class AssistantService {
    private final ChatClient client;
    String chat(String user, String msg) {
        return client.prompt().user(msg)
            .advisors(a -> a.param(CONV_ID, user))
            .call().content();
    }
}
```

~25 lignes · 2 classes · 1 builder chain

◆ LANGCHAIN4J-CDI / JAKARTA EE

```
// That's it. One interface.
@RegisterAIService(
    chatMemoryProviderSupplier
        = MemoryProvider.class,
    tools = WeatherTools.class,
    retrievalAugmentor = RagAugmentor.class)
public interface Assistant {
    @SystemMessage("You are helpful")
    String chat(@MemoryId String user,
        @UserMessage String msg);
}

// Inject anywhere. Done.
@Inject
Assistant assistant;
assistant.chat("yann", "Bonjour !");
```

~10 lignes · 1 interface · 0 passe-partout

- Declarative.
Injectable.
Portable. –
Just CDI doing
what CDI does
best.



Dependency injection is like medicine:

- Done properly, it doesn't hurt.
- Done incorrectly, you get a *NullPointerException*.
- And with CDI 4.1, we even have injection ... at compile time.



CDI 4.1: Build Compatible Extensions

PORTABLE EXTENSION

Treatment at runtime

WildFly, Payara, GlassFish, Liberty

```
void processAnnotatedType(  
    @Observes  
    ProcessAnnotatedType<?> pat) {  
    // Scan au runtime  
}
```

BUILD COMPATIBLE EXT.

Processing at compilation

Quarkus, Helidon

```
@Enhancement(  
    types = RegisterAIService.class,  
    withAnnotations = RegisterAIService.class)  
void enhance(ClassConfig cfg) {  
    // Enrichissement au build  
}
```

- Langchain4J-CDI provides both
 - It's transparent to you. 🎉
- It also runs on Quarkus. 😊



Under the hood

1. The extension scans for `@RegisterAIService`
2. For each interface found, it reads the attributes: `chatModelName`, `tools`, `chatMemoryProviderName`, `retrievalAugmentorName`, etc.,
3. The extension, then, creates a CDI **synthetic bean**:
 - Proxy `LangChain4J` coated in a managed CDI bean.
4. `LangChain4J` components are resolved via a **SPI** (`LLMConfig`):
 - `dev.langchain4j.cdi.plugin.<cdi-bean-name>.class=`
 - Implementation `MicroProfile Config` provided by `langchain4j-cdi-config`
5. Inject using `@Inject` →  It works!



Configuration: Microprofile Config



```
# Model configuration
dev.langchain4j.cdi.plugin.my-model.class=dev.langchain4j.model.openai.OpenAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${OPENAI_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=gpt-4o

# Memory configuration
dev.langchain4j.cdi.plugin.my-memory.class=dev.langchain4j.memory.chat.MessageWindowChatMemory
dev.langchain4j.cdi.plugin.my-memory.config.max-messages=20

# Linking to other beans: lookup:@default, lookup:@all, lookup:name
dev.langchain4j.cdi.plugin.my-retriever.config.embeddingStore=lookup:@default
```

- No code – Just properties.
 - **It's 2026!**



DEMO 1

@RegisterAIService IN ACTION

- ✓ Simple chat with @RegisterAIService
- ✓ Streaming: real-time responses 🎬
- ✓ Vision: image analysis 📷



The AI service interface

```
package com.example.demo1;

import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.cdi.RegisterAIService;

@RegisterAIService(chatModelName = "my-model")
public interface ChatAssistant {

    @SystemMessage("""
        You are a Viking skald performing in the great hall.
        You tell jokes about warriors, longships, axes, and the epic chaos of Viking raids. Your humor is sharp,
        self-deprecating, and full of Norse bravado.
        """)
    String chat(@UserMessage String userMessage);
}
```



The REST endpoint

```
• • •
@Path("/chat")
@ApplicationScoped
public class ChatResource {

    @Inject
    ChatAssistant assistant; // <- class CDI injection!

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String chat(String message) {
        return assistant.chat(message);
    }
}
```

- No LangChain4j dependency in your business code!



Streaming: Real-time responses



```
@RegisterAIService(streamingChatModelName = "my-streaming-model")
public interface ChatAssistantStreaming {

    @SystemMessage("You are a creative assistant...")
    TokenStream chatStream(@UserMessage String userMessage);
}
```



Streaming: Real-time responses



```
// In the Jakarta REST (formerly JAX-RS) endpoint with Server-Sent Events
@GET
@Path("/stream")
@Produces(MediaType.SERVER_SENT_EVENTS)
public void chatStream(@QueryParam("message") String message, @Context SseEventSink sink, @Context Sse sse) {
    TokenStream tokenStream = streamingAssistant.chatStream(message);

    tokenStream
        .onPartialResponse(token -> sink.send(sse.newEvent("token", token)))
        .onCompleteResponse(resp -> { sink.send(sse.newEvent("done", ""));
                                     sink.close(); })
        .onError(error -> { sink.send(sse.newEvent("error", error.getMessage()));
                           sink.close(); })
        .start();
}
```

- ChatGPT effect: the tokens are coming as you go!





Video: image analysis

```
@WebServlet("/uploadServlet")
@MultipartConfig
public class ImageAnalyzerServlet extends HttpServlet {

    @Inject
    @Named("vision-model")
    ChatModel visionModel; // <- A simple ChatModel!

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ... {
        Part file = request.getPart("file");

        UserMessage userMessage = UserMessage.from(
            TextContent.from("Describe this image in detail"),
            ImageContent.from(encodeBase64(file.getInputStream()), file.getContentType())
        );

        ChatResponse answer = visionModel.chat(userMessage);
        response.getWriter().write(answer.aiMessage().text());
    }
}
```

- **Multimodal** = text + image in the same message



Configuration: Streaming & Vision



```
# Streaming model configuration
dev.langchain4j.cdi.plugin.my-streaming-model.class=\
dev.langchain4j.model.openai.OpenAiStreamingChatModel
dev.langchain4j.cdi.plugin.my-streaming-model.config.api-key=${OPENAI_API_KEY}
dev.langchain4j.cdi.plugin.my-streaming-model.config.model-name=gpt-4o

# Configuring the vision model (via CDI @Named)
dev.langchain4j.cdi.plugin.vision-model.class=\
dev.langchain4j.model.ollama.OllamaChatModel
dev.langchain4j.cdi.plugin.vision-model.config.base-url=http://localhost:11434
dev.langchain4j.cdi.plugin.vision-model.config.model-name=llama3.2-vision
dev.langchain4j.cdi.plugin.vision-model.config.timeout=PT5M
```

- **streamingChatModelName** for streaming @Named + ChatModel for direct model injection



What we just saw

- **Classic chat**

- @RegisterAIService
+ @Inject
= Skald Viking 🗡️🎤

- **Streaming**

- TokenStream
+ Server-Sent
Events
= Viking Saga Live 🎵

- **Multimodal vision**

- TextContent +
ImageContent
= Analysis of
images with AI

All this with CDI. Zero plumbing code. 🍷



An LLM without memory ...

It's like a developer on Monday morning.

He forgot everything he knew on Friday.

Thankfully, we have ChatMemory!



Conversational memory



```
@RegisterAIService(chatModelName = "my-model", chatMemoryProviderName = "my-memory")
public interface ChatAssistant {
    String chat(@MemoryId String sessionId, @UserMessage String message);
}
```

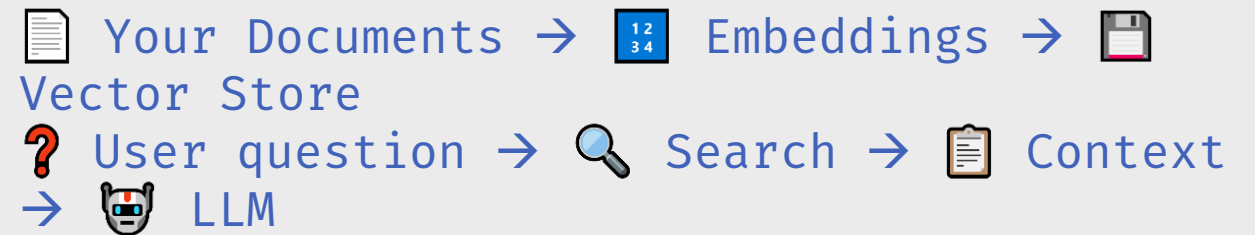


```
# microprofile-config.properties
dev.langchain4j.cdi.plugin.my-memory.class=dev.langchain4j.memory.chat.MessageWindowChatMemory
dev.langchain4j.cdi.plugin.my-memory.config.max-messages=20
```

@MemoryId → one memory per session / user



RAG: Retrieval Augmented Generation





RAG



```
@RegisterAIService(chatModelName = "my-model", contentRetrieverName = "my-rag")
public interface KnowledgeAssistant {

    @SystemMessage("Answer based on the provided context")
    String answer(@UserMessage String question);
}
```



RAG (ContentRetriever)

```

● ● ●
@ApplicationScoped
public class KnowledgeBaseProvider {
    @Inject ConferenceRepository repository;

    @Produces @Named("my-rag") @ApplicationScoped
    public ContentRetriever contentRetriever() {
        // EmbeddingModel + InMemoryEmbeddingStore + ingestion
        return EmbeddingStoreContentRetriever.builder()
            .embeddingStore(store)
            .embeddingModel(embeddingModel)
            .maxResults(3).build();
    }
}
```




Tools: Function calling

```

@ApplicationScoped
public class ExpeditionTools {

    @Inject
    ExpeditionService expeditionService;

    @Tool("List available Viking expeditions for a given date")
    public List<Expedition> listExpeditions(@P("Date in yyyy-MM-dd format") String date) {
        return expeditionService.getExpeditions(LocalDate.parse(date));
    }

    @Tool("Enroll a warrior in an expedition")
    public Enrollment enrollWarrior(@P("Warrior name") String name, @P("Desired date") String date) {
        return expeditionService.enroll(name, LocalDate.parse(date));
    }
}

```

Tools are **permanent beans** → injection of dependencies in the tools! 🍌



**It's working
locally!**

But in prod, the LLM
decided
to answer in Klingon.

And your endpoint timeout after 30 seconds.
We need safeguards!



Microprofile Fault Tolerance

```
@RegisterAIService(chatModelName = "my-model")
public interface ResilientAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    @Timeout(value = 30, unit = ChronoUnit.SECONDS)
    @Fallback(fallbackMethod = "fallbackChat")
    @CircuitBreaker(requestVolumeThreshold = 5,
        failureRatio = 0.5)
    String chat(@UserMessage String message);

    default String fallbackChat(String message) {
        return "Service temporarily unavailable. "
            + "Please try again in a moment.";
    }
}
```



Microprofile Telemetry

TRACES

OpenTelemetry native

- ☒ Latency LLM
- ☒ Tokens in/out
- ☒ Calls tools
- ☒ RAG chain

METRICS

Grafana LGTM

- ☒ Cost per request
- ☒ Error rate
- ☒ Response time
- ☒ Circuit breaker



```
<!-- Simply add the dependency -->
<dependency>
  <groupId>dev.langchain4j.cdi.mp</groupId>
  <artifactId>langchain4j-cdi-telemetry</artifactId>
</dependency>
```



OpenTelemetry configuration



```
# MicroProfile OpenTelemetry - OTLP Exporter
otel.exporter.otlp.endpoint=http://localhost:4318
otel.service.name=demo-2-langchain4j-cdi
otel.traces.exporter=otlp
otel.metrics.exporter=otlp

# LangChain4j-CDI Telemetry - Listeners
dev.langchain4j.cdi.plugin.my-model.config.listeners=\
    dev.langchain4j.cdi.telemetry.SpanChatModelListener,\
    dev.langchain4j.cdi.telemetry.MetricsChatModelListener
```

Listeners automatically capture all AI interactions



Grafana LGTM: only one order



```
# Start the complete observability stack  
podman run -p 3000:3000 -p 4317:4317 -p 4318:4318 --rm -ti grafana/otel-lgtm
```

- 🎁 **LGTM** = Loki + Grafana + Tempo + Mimir
 - ✓ **Tempo**: trace storage
 - ✓ **Loki**: aggregation of logs
 - ✓ **Mimir**: time-series metrics
 - ✓ **Grafana**: unified visualization
 - ✓ **OTLP Collector**: integrated on ports 4317/4318

Open <http://localhost:3000> → Explore → Tempo 



DEMO 2

Viking expedition assistant

Memory + RAG + Tools + Fault Tolerance + Telemetry

-  Memory per session (@MemoryId)
-  RAG on shipments (contentRetrieverName)
-  Registration tools (@Tool+ CDI)
-  Fault Tolerance via live coding!

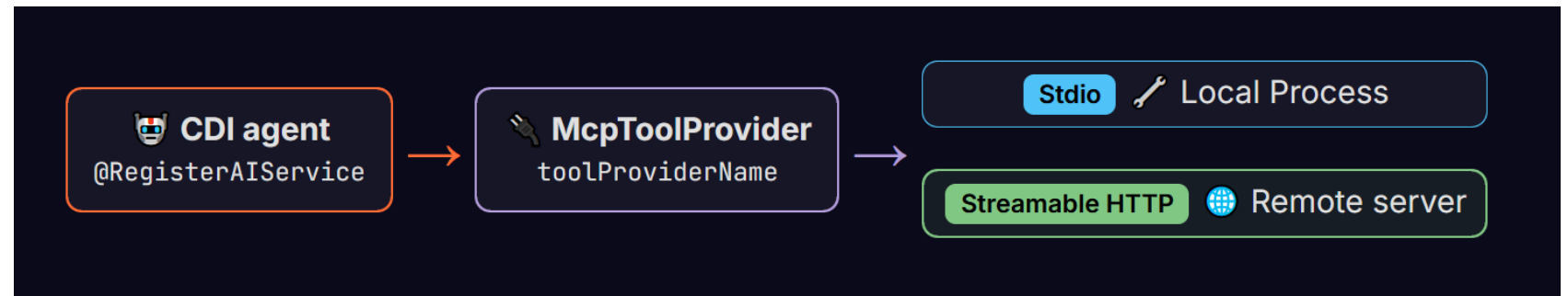


Langchain4J-CDI modules

- **Core:** langchain4j-cdi
 - API + annotations
- **Agentic:** langchain4j-cdi-agentic
 - @RegisterSequenceAgent, @RegisterLoopAgent, etc.
- **Portable:** langchain4j-cdi-portable-ext
 - Wildfly, Payara, Glassfish, Liberty
- **Build:** langchain4j-cdi-build-compatible-ext
 - Quarkus, Helidon
- **Resiliency:** langchain4j-cdi-fault-tolerance
 - @Retry, @Timeout, @CircuitBreaker
- **Observability:** langchain4j-cdi-telemetry
 - OpenTelemetry, Metrics
- **Coming soon?** Langchain4j-cdi-security
 - JWT, RBAC for agents?



MCP: Model Context Protocol



```
@RegisterAIService(chatModelName = "my-model", toolProviderName = "mcp-tools")
public interface AgentWithMCP {
    String execute(@UserMessage String task);
}
```



MCP: Model Context Protocol



```
# MicroProfile Config → McpToolProvider via HTTP streamable  
dev.langchain4j.cdi.plugin.mcp-tools.class=dev.langchain4j.mcp.McpToolProvider  
dev.langchain4j.cdi.plugin.mcp-tools.config.transport-type=http  
dev.langchain4j.cdi.plugin.mcp-tools.config.url=http://localhost:8081/mcp
```

Zero plumbing code -- everything goes through the properties 🌐



DEMO 3

Hnefatafl at the Grand Thing with MCP



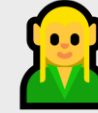
, the Jarl who animates the Hnefatafl at the Grand Thing

- ✓ External HTTP SSE MCP Server for Dice Launches
- ✓ AI Agent with @RegisterAIService + McpToolProvider
- ✓ Configuration via Microprofile Config!



Flow: Rune casting via MCP SSE

1. **Warrior**: "Launch the runes"
2. **Jakarta REST Endpoint** → `HnefataflJarlAI.play()`
3. **LLM (Ollama)** → decides to use `roll(numberOfDice=2)`
4. **McpToolProvider** → JSON-RPC → **MCP Runes Server** (HTTP SSE on `localhost:8090`)
5. **Server MCP** launches 2d6 → returns `{"rolls": [4, 3], "total": 7}` via SSE
6. **LLM** analyzes the result → identifies "Favor of Odin" → responds to the warrior
7. **Ragnar the Skald**: "BY ODIN! Odin's favor! The warrior triumphs! 🗡️🛡️"



The skald sang in English.

...with elves *and* dwarfs in the saga.

The LLM is non-deterministic. He does what he wants.

The solution: **Guardrails** – the safeguards of your AI agents



Guardrails: validate the input and output

```

@RegisterAIService(
    chatModelName      = "my-model",
    inputGuardrailNames = {"english-input", "fantasy-input"},
    outputGuardrailNames = {"english-output", "fantasy-output"})
public interface SkaldAssistant {

    @SystemMessage("You are a Viking skald who composes epic songs in ENGLISH...")
    String composeSong(@UserMessage String request);
}

```

• Input Guardrail

- Filters queries *before* LLM
- ☒ Valid language?
- ☒ Suitable content?
- ☒ No forbidden words?

• Output Guardrail

- Validates the answers *after* the LLM
- ☒ Language of the response?
- ☒ Content compliant?
- ☒ Expected format?



Guardrails = Named CDI Beans

```

@ApplicationScoped
@Named("english-input") // <- Name referenced in @RegisterAIService
public class EnglishInputGuardrail implements InputGuardrail {

    private LanguageDetector detector;

    @PostConstruct
    void init() {
        detector = new OptimizeLangDetector().loadModels();
    }

    @Override
    public InputGuardrailResult validate(UserMessage userMessage) {
        String text = userMessage.singleText();
        LanguageResult result;
        synchronized (detector) { // <- Thread-safe !
            result = detector.detect(text);
        }
        if (result.isReasonablyCertain() && !"en".equals(result.getLanguage())) {
            return failure("Only English is accepted!");
        }
        return success();
    }
}

```

- Apache Tika for language detection
 - benefit of the doubt for short or ambiguous texts



DEMO 4

Skald Viking with Guardrails

- 🛡️ **4 guardrails** around a Viking song composition agent
- ✅ Input Guardrail: English language only
- ✅ Input Guardrail: no fantastic breeds in the request
- ✅ Output Guardrail: the generated song is well in English
- ✅ Output Guardrail: no dwarfs or elves in the song

Guardrails are CDI beans – injection of dependencies possible! 🎉



Output guardrail: Validate the answer

```

@ApplicationScoped
@Named("fantasy-output")
public class NoFantasyRacesOutputGuardrail implements OutputGuardrail {

    private static final Set<String> FORBIDDEN_WORDS =
        Set.of("dwarf", "dwarves", "elf", "elves");

    @Override
    public OutputGuardrailResult validate(AiMessage responseFromLLM) {
        String text = responseFromLLM.text().toLowerCase();

        for (String word : FORBIDDEN_WORDS) {
            if (text.contains(word)) {
                return failure("The skald's song must not evoke"
                    + "dwarves or elves! This is a pure Viking saga.");
            }
        }
        return success();
    }
}

```

- The OutputGuardrail receives the complete AiMessage – same pattern as the InputGuardrail



A2A: Agent-to-agent protocol

- MCP

Agent ↔ Tools

An agent calls external *tools* (databases, APIs, files ...)

- A2A

Agent ↔ Agent

An agent **delegates** to other agents specialized, each on their own server



Microservices, versioned AI agents



A2A Architecture: Story forge

- **Creative Writer**
Agent A2A (Port 8080)
`@RegisterAIService + AgentCard`
- **Style Scorer**
Agent A2A (Port 8081)
`@RegisterAIService + AgentCard`
- **Orchestrator** (Port 8082)
LangChain4j Agentic :
sequence + loop

 Skald forges the saga →  Judge evaluates (score < 0.8?) →  Master skald rewritten →  Loop



Server A2A = AgentCard + Executor

```

@RegisterAIService(chatModelName = "ollama", scope = ApplicationScoped.class)
public interface CreativeWriter {
    @UserMessage("Forge a Norse saga around the topic {{topic}}...")
    String generateStory(@V("topic") String topic);
}

```

@RegisterAIService +
AgentCard CDI producer
+ AgentExecutor = full
A2A server

```

@Produces @PublicAgentCard
public AgentCard agentCard() {
    return new AgentCard.Builder()
        .name("Creative Writer")
        .skills(List.of(new AgentSkill.Builder()
            .id("creative_writer").build()))
        .preferredTransport(TransportProtocol.JSONRPC.asString())
        .build();
}

```



Orchestrator: Sequence + Loop

```
A2ACreativeWriter writer = AgenticServices
    .a2aBuilder(writerUrl, A2ACreativeWriter.class)
    .outputKey("story").build();

UntypedAgent reviewLoop = AgenticServices.loopBuilder()
    .subAgents(styleScorer, styleEditor)
    .maxIterations(5)
    .exitCondition(scope -> scope.readState("score", 0.0) >= 0.8)
    .build();

StyledWriter pipeline = AgenticServices
    .sequenceBuilder(StyledWriter.class)
    .subAgents(writer, reviewLoop)
    .outputKey("story").build();
```

a2aBuilder → remote agent via A2A | loopBuilder → loop with output condition |
sequenceBuilder → sequential pipeline



DEMO 5

Story Forge – A2A Orchestration

- ♥ 3 AI services communicating via A2A protocol
- ✓ **Writer:** forges a Nordic saga on a subject
- ✓ **Style Scorer:** judges the alignment of the saga with a style
- ✓ **Orchestrator:** sequence + loop revision (score ≥ 0.8)
- ✓ **Style Editor** : local rewrite if the score is insufficient

AI microservices, wired with CDI and LangChain4j Agentic 🚀



AgentExecutor: receive A2A requests

```
@Produces
public AgentExecutor agentExecutor() {
    return new AgentExecutor() {
        @Override
        public void execute(RequestContext context, EventQueue eventQueue) throws JSONRPCError {
            TaskUpdater updater = new TaskUpdater(context, eventQueue);
            if (context.getTask() == null) { updater.submit(); }
            updater.startWork();

            String userMessage = extractTextFromMessage(context.getMessage());
            String response = creativeWriter.generateStory(userMessage);

            updater.addArtifact(List.of(new TextPart(response, null)),
                null, null, null);
            updater.complete();
        }
    };
}
```

The AgentExecutor is a CDI producer – pattern identical to the AgentCard



What we saw today?

-  A CDI extension that transforms interfaces into AI agents
-  @Inject an agent as simply as an EntityManager
-  100% MicroProfile Config configuration
-  Memory, RAG, Tools, MCP -- all injectable
-  + Telemetry out of the box
-  -- validate entry and exit with permanent contracts
-  -- multi-agent orchestration with LangChain4j Agentic
-  Compatible with **all** CDI servers



Join us!

- ✨ github.com/langchain4j/langchain4j-cdi
- 🐉 Welcome issues -- even crazy feature requests
- 🔗 PRs encouraged -- review quickly (promised)
- 💬 GitHub Discussions -- for questions
- Thank you also to **Yann Blazart** & **Emmanuel Hugonnet**, co-creator 🙏

This slide is the English translation of the slides provided by my co-creator at Devovx France 2026.

<https://github.com/ehsavoie/langchain4j-cdi-lab-2026>



Feedback



Langchain4J CDI in Action



Questions?

"Will AI replace Java developers?"

No, but a Java developer with AI
Will replace the one without.

And with CDI, AI is just another @Inject.





THANK YOU



Buhake Sindi

x: @EliteGentleman

Other socials: @buhake.sindi

<https://linktr.ee/theEliteGentleman>



<https://github.com/TheEliteGentleman/langchain4j-cdi-lab-jcon-slovenia-2026>

Slides and code

